

FRC Java / Command-Based Robot — Intro Session Notes

Topic: Introduction to FRC Java object-oriented programming (command-based framework) **Project context:** Building a "ChairBot" (a drivable chair robot) as a simple teaching project. The goal was to learn the fundamentals of subsystems, motor controllers, and controller bindings.

Quick context for what you missed: We coded a simple drivetrain from scratch in WPILib to learn the fundamentals — motor controllers, subsystems, methods, and controller bindings. This was the introductory session, so it's all fundamentals. There will be more meetings (PID tuning, control theory, possibly reprogramming the main robot). Below is everything we covered, in order.

1. Setting Up a New WPILib Project

1. Find the **WPILib logo** (the red/black "W") in the **top-right corner** of VS Code. If you don't see it, create any new file first and it should appear.
 2. Click it to open the **WPILib Command Palette**.
 3. Type and select **"Create a new project."**
 4. Fill in the project creator page:
 - **Project type:** Template
 - **Language:** Java
 - **Base/template:** Command Robot (command-based)
 - **Folder:** select or create a folder (note: keep it out of OneDrive to avoid sync issues)
 - **Project name:** e.g., `ChairBot`
 - **Desktop support:** NOT needed (that's only for simulation)
 5. Click **Generate Project**, then open it in the current or a new window.
 6. Let Gradle run its build ("started a daemon" / build process). Wait until you see **"BUILD SUCCESSFUL"** and **"Java ready"** at the bottom-left before continuing.
-

2. Project Structure — Where You Actually Work

Navigate to: `src` → `main` → `java` → `frc` → `robot`

You can **ignore everything else** unless you're doing advanced/low-level work.

Inside the `robot` folder, the files that matter:

File	What it's for
<code>Robot.java</code>	Don't touch unless doing low-level things (test systems, etc.)
<code>RobotContainer.java</code>	Where you wire up controllers and bindings (configure bindings area)
<code>subsystems/</code>	Where you control hardware (motors). You'll work here a lot.
<code>commands/</code>	Command classes (not needed for this simple project)
<code>Constants.java</code>	Store values you reuse but rarely change (e.g., port numbers)

Key idea: For a simple project like this, you don't even need `commands` — you can do everything with **subsystems**.

3. Subsystems — The Basics

Open the `ExampleSubsystem` to see the standard structure:

- **Constructor** — `public ExampleSubsystem()`
- `periodic()` — runs **every 20 milliseconds**, continuously, unless a condition pauses it (rarely happens). Think of it like Arduino/Romi `loop()`.
- `simulationPeriodic()` — only for simulations, ignore for us.

What a subsystem does: It's what *interacts with the motors*. When you want to turn a motor on, you do it through a subsystem.

Making our drivetrain subsystem

1. **Duplicate** `ExampleSubsystem` (copy/paste).
2. **Rename** it to something like `DrivetrainSubsystem`.
3. Inside, replace any red squiggly-underlined references (the old class name) with the new file name. The class line should read `public class DrivetrainSubsystem`.
 - Note: some VS Code setups auto-rename these for you (an extension does it). Others (like on the older laptops) require doing it by hand.
 - **Red squiggles** = errors. **Yellow** = a warning, usually something declared but not used — not a blocker.

4. Motor Controllers

Important gotcha — Jaguars are discontinued

We originally tried to use **Jaguar** motor controllers (`import edu.wpi.first...motorcontrol.Jaguar`), but **they've been discontinued / removed** from current WPILib, so the import fails. (This is why the actual ChairBot will reuse last year's code, which still uses Jaguars — we'll just edit that code for the real robot. For learning purposes, we used SparkMax instead.)

Installing a vendor library (RevLib for SparkMax)

To use **SparkMax** motors, you need the **RevLib** vendor dependency:

1. Click the **WPILib logo at the bottom-left** ("WPILib Vendor Dependencies").
2. Under **Available dependencies**, scroll to **REVLlib** and install it.
3. Confirm it shows up at the top under installed dependencies.

┆ This same process is how you add any vendor motor-controller library.

Declaring motors

At the **top of the class** (right under `public class DrivetrainSubsystem`), declare your motor variables.

The syntax follows normal Java variable creation — **type** → **name** → **value**:

```
java

// type      name      value (new instance)
SparkMax leftMotor1 = new SparkMax(2, MotorType.kBrushless);
```

- `SparkMax` is provided automatically once RevLib is installed — when you type `SparkMax` and let autocomplete fill it in, the import is added for you.
- First argument = the **device ID** (don't make them all `0` — give each a unique ID).
- Second argument = the **motor type**: `MotorType.kBrushless`.
 - If the motor type doesn't auto-import, retype `Mo...` and pick `MotorType` from autocomplete.

Brushed vs. brushless: Brushed motors are the older type; **brushless** are the newer ones we actually use (e.g., Krakens). We almost always use brushless.

How many motors / why four

The ChairBot has **two sides**, with **two wheels per side**, and **two motors driving each side**. So we declare **four motor variables total**: two left, two right.

```
java
```

```
SparkMax leftMotor1 = new SparkMax(2, MotorType.kBrushless);  
SparkMax leftMotor2 = new SparkMax(3, MotorType.kBrushless);  
SparkMax rightMotor1 = new SparkMax(4, MotorType.kBrushless);  
SparkMax rightMotor2 = new SparkMax(5, MotorType.kBrushless);
```

(IDs are examples — just make each unique.)

Reminder on syntax parallel: `SparkMax leftMotor1 = new SparkMax(...)` is structurally the same as `int num = 1;` — variable type, name, then a value/new instance. That's all that's happening.

5. Methods to Control the Motors

At the **bottom of the subsystem**, create methods to set motor speeds.

- `public void` because we're **not returning anything**.
- Takes in a `double speed` parameter.
- **Speed range is -1.0 to 1.0** (negative = reverse, positive = forward).
- Use the fundamental `.set()` call on each motor.

```
java
```

```
public void setLeftMotors(double speed) {  
    leftMotor1.set(speed);  
    leftMotor2.set(speed);  
}  
  
public void setRightMotors(double speed) {  
    rightMotor1.set(speed);  
    rightMotor2.set(speed);  
}
```

Big takeaway: `motor.set(speed)` is *the* fundamental call. If you understand `.set()`, you're already about a third of the way to making a robot move. Note that a method does nothing until it's actually **called** — declaring it just defines the behavior.

Subsystem structure recap: variables (your motors) at the top → methods (set-left / set-right) at the bottom that take in a value sent from `RobotContainer` and apply it to the motors.

6. Controllers in RobotContainer

Controllers are wired up in `RobotContainer` (at the top, where things are declared).

Two controller types — know the difference

This distinction is important:

Type	What it gives you	When to use
<code>PS5Controller</code> (plain)	The state of buttons (is X pressed? is it 50% pressed?). Returns booleans/values. Does not run commands.	When you need to <i>read</i> state — e.g., "is the align button held? then lower drive speed by hand."
<code>CommandPS5Controller</code>	Runs a command when a button state changes (pressed/released). Doesn't hand you the raw state.	The common one in command-based robots. Used most of the time.

```
java
// Plain — reads state:
PS5Controller joystick = new PS5Controller(0);

// Command-based — triggers commands:
CommandPS5Controller joystick2 = new CommandPS5Controller(0);
```

The number (`0`) is the **port**.

Ports & the Driver Station

- The port number maps to a slot in the **Driver Station application** (not the laptop itself — the FRC Driver Station app on Windows).
- In the Driver Station's **controller/USB section**, you **drag and drop** your plugged-in controller into a port (usually **port 0**).
- When the code initializes the controller variable with port `0`, it reads inputs from whatever is assigned to port 0 in the Driver Station.
- **Ports identify the controller, not individual buttons.** Which button you read is handled in code (below), not by ports.

7. Constants — Why Use Them

In the project template you'll see something like

`OperatorConstants.kDriverControllerPort` instead of a raw `0`.

- A **constant** is just a stored value (e.g., `kDriverControllerPort = 0`) in `Constants.java`.
 - **Why bother instead of typing `0`**? If multiple controllers/files use that port and you later need to change it, you'd have to hunt down and edit every `0` by hand. With a constant, you change it **once** in `Constants.java` and it updates everywhere it's referenced.
 - This is the textbook use-case for the constants file: a value used in multiple places that rarely changes.
-

8. Binding Buttons to Actions

In `RobotContainer`'s **configure bindings** area, you bind a button to an action.

Pattern

```
controller.button().condition(commandToRun)
```

```
java
joystick2.cross().onTrue( /* command here */ );
```

Conditions you'll care about (ignore the rest):

- `onTrue` — fires once when the button becomes pressed
- `onFalse` — fires once when released
- `whileTrue` — runs while held down
- `whileFalse` — runs while not held

Side note: `cross()` is the PS5 "X" button. (Yes, there was confusion between "the cross" button and "lacrosse" on the call.)

Initialize the subsystem first

You **can't call methods straight off the class name** — these aren't static methods. You must create an **instance** of the subsystem at the top of `RobotContainer`:

```
java
DrivetrainSubsystem mDrivetrain = new DrivetrainSubsystem();
```

Static vs. non-static: A static method is called on the class itself, e.g., `Math.abs()` or `Math.pow()` — no instance needed. Our subsystem methods are **not** static, so you need an instance (`mDrivetrain`) to call `mDrivetrain.setLeftMotors(...)`.

Running a subsystem method on a button — InstantCommand + lambda

Most of the time you won't have a pre-made command — you'll just have subsystem methods like `setRightMotors`. To run one on a button press, wrap it in an `InstantCommand` using a **lambda**:

```
java
joystick2.cross().onTrue(
    new InstantCommand(() -> mDrivetrain.setLeftMotors(1))
);
```

What a lambda is: a small function you can write inline, on the spot, in a single line of code `((() -> ...))`.

What `InstantCommand` does: Commands normally need an ending condition. An `InstantCommand` runs once and **immediately ends**. That makes it perfect for **sequences** — e.g., turn on rollers (instant) → then the next step fires immediately after, like in a shoot or intake sequence.

9. Full Signal Flow (Controller → Motor)

Putting it all together, here's how a press travels through the code:

1. Initialize a `CommandPS5Controller` (mapped to a Driver Station port).
2. Button is pressed → its state changes to **true**.
3. `onTrue` fires an `InstantCommand`.
4. The `InstantCommand`'s lambda calls `mDrivetrain.setLeftMotors(1)`.
5. Inside `setLeftMotors`, `speed` is set to `1`, and `.set(1)` is applied to each left motor.
6. All left motors now run at full power and **keep running at that speed** until it's changed.

Repeat the same pattern with different buttons and different speeds/methods for full control.

That's the whole fundamental loop. The advanced stuff (PID, feedforward, control theory) sits on top of this — but if you understand the above, you could technically make the robot move (just not smoothly yet).

10. ChairBot Project Decisions

- **Control method:** We chose a **controller** (over foot-operated limit switches or a voice-command experiment) because it matches what we use on the real robot. A driver will sit and drive with a controller.
 - **Real ChairBot code:** Because Jaguars are discontinued, the actual ChairBot will reuse **last year's GitHub code** (which already supports Jaguars), edited as needed. The SparkMax version we built today was for learning.
 - **Voice commands (Enzo's side experiment):** Got a working offline prototype using **Vosk** (a ~40 MB offline speech-recognition model that runs locally — e.g., on a Raspberry Pi or the Driver Station laptop, no internet/API needed). The standard FRC pattern is to recognize speech on a computer and send results to the robot over **NetworkTables**. It works, but there are **liability concerns** (one drive motor drifts right and needs constant correction, and the only safety feature is having to hold something while issuing a command) — so it may not make it onto the real robot.
-

11. Concepts to Review / Coming Up Later

- **PID tuning and feedforward** — the "hard part" of control theory; covered in a future session once we have things that move.
 - **Programming switches** — chained conditionals (like a long if/else): check conditions in order, run the matching one, else fall to a default. Functionally interchangeable with if-statements.
 - **Simulation** — we experimented with simulating code this year but couldn't get the tools to work reliably, so we currently just wait for the physical robot. Possible future topic.
 - **Future sessions** may include reprogramming the main robot from scratch.
-

Key Terms Cheat Sheet

- **Subsystem** — class that interacts with hardware (motors). Variables at top, methods at bottom.
 - `periodic()` — runs every 20 ms continuously.
 - **SparkMax** — the motor controller we used (needs RevLib installed).
 - **Brushless** (`kBrushless`) — the newer motor type we use; brushed is the old one.
 - `.set(speed)` — the fundamental command to drive a motor; speed is -1.0 to 1.0 .
 - `PS5Controller` — reads button *state*.
 - `CommandPS5Controller` — runs *commands* on button state changes (used most often).
 - **Constant** — a reusable stored value (e.g., a port number) you change in one place.
 - **Lambda** (`() -> ...`) — an inline single-line function.
 - `InstantCommand` — a command that runs once and immediately ends; great for sequences.
 - **NetworkTables** — how a computer sends data to the robot (used in the voice-command experiment).
-

One tip that came up and is worth repeating: don't be afraid to say when something doesn't make sense. Keep poking at it until it clicks — that matters more than nodding along.